

PRD — PulsePass: Implementación de la Capa de Servicios

1. Información general

Producto: PulsePass

Módulo: Capa de Servicios

Versión: 1.0

Tecnologías objetivo: Java 21, Spring Boot 4, Spring Data JPA, MapStruct, JUnit 5, Mockito y AssertJ

Tipo: Product Requirements Document (PRD)

2. Resumen ejecutivo

PulsePass es una plataforma para descubrir eventos y gestionar entradas para conciertos, festivales, conferencias, eventos universitarios, competencias deportivas y actividades culturales.

La capa de persistencia ya permite almacenar y consultar:

- venues;
- eventos;
- artistas;
- usuarios;
- perfiles de usuario;
- tickets.

El objetivo de esta fase es implementar la **capa de servicios**, responsable de aplicar reglas de negocio, coordinar repositories, controlar transacciones, transformar entidades a DTOs y producir errores de dominio claros.

La capa Service debe convertirse en la frontera entre las futuras capas de exposición y el modelo persistente.

3. Objetivos

La capa de servicios deberá permitir:

1. consultar venues;
2. crear y consultar eventos;
3. publicar eventos;
4. asociar artistas a eventos;
5. registrar y consultar usuarios;
6. comprar tickets;
7. cancelar tickets;
8. marcar tickets como usados;
9. consultar tickets por usuario y evento;
10. validar capacidad;

11. validar edad mínima;
 12. controlar transiciones de estado;
 13. retornar DTOs sin exponer entidades JPA.
-

4. Alcance

4.1 Incluido

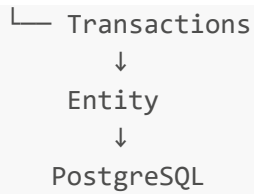
- Interfaces Service.
- Implementaciones `@Service`.
- DTOs request/response con Java `record`.
- MapStruct.
- Reglas de negocio.
- `Optional`.
- Excepciones personalizadas.
- `@Transactional`.
- `@Transactional(readOnly = true)`.
- Programación funcional cuando aporte claridad.
- Unit tests con JUnit 5.
- Mockito.
- AssertJ.

4.2 Fuera de alcance

- Controllers REST.
 - Spring Security.
 - JWT.
 - Frontend.
 - Pasarela de pagos.
 - Notificaciones.
 - Kafka.
 - Expiración de reservas.
 - QR reales.
 - Tests de integración de Service con PostgreSQL.
 - Testcontainers para unit tests.
-

5. Arquitectura esperada

```
Controller
  ↓ futuro
Service Interface
  ↓
Service Implementation
  ├── Repository
  ├── Mapper
  └── Business Rules
```



La separación esperada es:



6. Modelo de dominio existente

Entidades disponibles:

Venue
Event
Artist
User
UserProfile
Ticket

Enums:

EventCategory
EventStatus
TicketType
TicketStatus

Relaciones principales:

Venue 1 — N Event
Event N — M Artist
User 1 — 1 UserProfile
User 1 — N Ticket
Event 1 — N Ticket

7. Principios de diseño

SRV-001 — No exponer entidades

Los contratos públicos de Service no deben retornar directamente entidades como **Event**, **Ticket**, **User**, **Venue** o **Artist**.

Deben utilizar DTOs.

SRV-002 — Inyección por constructor

Las implementaciones deberán usar dependencias **final** e inyección por constructor.

SRV-003 — Contratos mediante interfaces

Cada servicio principal deberá tener interfaz e implementación.

Ejemplo:

```
EventService
EventServiceImpl
```

SRV-004 — Transacciones

Operaciones de escritura:

```
@Transactional
```

Operaciones de lectura:

```
@Transactional(readOnly = true)
```

cuando corresponda.

SRV-005 — Repositories sin reglas de negocio

Repository se ocupa del acceso a datos.

Service se ocupa de preguntas como:

```
¿Puede este usuario comprar una entrada para este evento?
```

8. Estructura sugerida

```
com.pulsepass
├── domain
├── repository
├── dto
│   ├── request
│   └── response
├── mapper
├── exception
└── service
    └── impl
```

DTOs sugeridos:

```
dto/request/
├── CreateEventRequest.java
├── RegisterUserRequest.java
└── PurchaseTicketRequest.java

dto/response/
├── VenueResponse.java
├── EventResponse.java
├── EventSummaryResponse.java
├── ArtistResponse.java
├── UserResponse.java
└── TicketResponse.java
```

9. Servicios requeridos

La primera versión debe implementar:

```
VenueService
EventService
ArtistService
UserService
TicketService
```

10. VenueService

10.1 Contrato mínimo

```
public interface VenueService {  
  
    VenueResponse findByCode(String code);  
  
    List<VenueResponse> findActiveVenues();  
}
```

10.2 Reglas

BR-VENUE-001

Si el venue solicitado no existe, lanzar:

```
ResourceNotFoundException
```

BR-VENUE-002

`findActiveVenues()` solo retorna venues con:

```
active = true
```

11. EventService

11.1 Contrato mínimo

```
public interface EventService {  
  
    EventResponse create(CreateEventRequest request);  
  
    EventResponse findByCode(String eventCode);  
  
    List<EventSummaryResponse> findPublishedEvents();  
  
    EventResponse publish(String eventCode);  
  
    EventResponse addArtist(  
        String eventCode,  
        Long artistId  
    );  
  
    List<EventSummaryResponse> findByArtist(  
        String stageName
```

```
}  
);
```

12. CreateEventRequest

Debe implementarse como **record**.

Campos:

```
eventCode  
name  
description  
category  
eventDate  
minimumAge  
venueCode
```

Ejemplo:

```
public record CreateEventRequest(  
    String eventCode,  
    String name,  
    String description,  
    EventCategory category,  
    LocalDateTime eventDate,  
    Integer minimumAge,  
    String venueCode  
) {}
```

13. EventResponse

Debe exponer como mínimo:

```
id  
eventCode  
name  
description  
category  
status  
eventDate  
minimumAge  
venueCode
```

```
venueName  
artists
```

No debe exponer directamente:

```
Venue  
Set<Artist>  
List<Ticket>
```

14. Reglas de creación de eventos

BR-EVENT-001 — Código único

No puede existir otro evento con el mismo `eventCode`.

Error:

```
DuplicateResourceException
```

BR-EVENT-002 — Venue obligatorio

`venueCode` debe corresponder a un venue existente.

Si no existe:

```
ResourceNotFoundException
```

BR-EVENT-003 — Venue activo

No se puede crear un evento en un venue inactivo.

Error:

```
BusinessRuleException
```

BR-EVENT-004 — Fecha válida

La fecha del evento debe ser futura al momento de creación.

BR-EVENT-005 — Estado inicial

Todo evento nuevo inicia en:

DRAFT

El request no controla este valor.

BR-EVENT-006 — Edad mínima

Debe cumplirse:

```
minimumAge >= 0
```

0 significa que no existe restricción de edad.

15. Publicación de eventos

La operación:

```
publish(eventCode)
```

debe realizar:

DRAFT → PUBLISHED

BR-EVENT-007

Solo se puede publicar un evento en estado **DRAFT**.

BR-EVENT-008

El evento debe seguir teniendo fecha futura.

BR-EVENT-009

El venue debe continuar activo.

No se permite publicar eventos:

PUBLISHED
SOLD_OUT

CANCELLED
FINISHED

16. Asociación Event ↔ Artist

`addArtist(eventCode, artistId)` deberá:

1. buscar evento;
2. buscar artista;
3. validar ambos recursos;
4. evitar duplicados;
5. asociar el artista;
6. guardar;
7. retornar `EventResponse`.

BR-EVENT-010

No se puede asociar dos veces el mismo artista al evento.

BR-EVENT-011

No se pueden agregar artistas a eventos:

CANCELLED
FINISHED

17. ArtistService

Contrato:

```
public interface ArtistService {  
  
    ArtistResponse findById(Long id);  
  
    ArtistResponse findByStageName(String stageName);  
  
    List<ArtistResponse> findActiveArtists();  
}
```

BR-ARTIST-001

Si el artista no existe:

ResourceNotFoundException

BR-ARTIST-002

`findActiveArtists()` solo retorna artistas activos.

18. UserService

Contrato:

```
public interface UserService {  
  
    UserResponse register(RegisterUserRequest request);  
  
    UserResponse findByEmail(String email);  
  
    UserResponse findByUsername(String username);  
}
```

19. RegisterUserRequest

Debe ser un `record` con:

```
username  
email  
firstName  
lastName  
phone  
city  
birthDate
```

20. Reglas de usuario

BR-USER-001

`username` debe ser único.

Error:

DuplicateResourceException

BR-USER-002

El email debe ser único ignorando mayúsculas/minúsculas.

BR-USER-003

Todo usuario nuevo inicia con:

```
active = true
```

BR-USER-004

El registro debe crear `User` y `UserProfile` dentro de la misma transacción.

BR-USER-005

`birthDate` no puede ser futura.

21. TicketService

Contrato:

```
public interface TicketService {

    TicketResponse purchase(
        PurchaseTicketRequest request
    );

    TicketResponse findByCode(
        String ticketCode
    );

    List<TicketResponse> findByUserEmail(
        String email
    );

    List<TicketResponse> findPaidTicketsByEvent(
        String eventCode
    );

    TicketResponse cancel(
        String ticketCode
    );
}
```

```
TicketResponse markAsUsed(  
    String ticketCode  
);  
}
```

22. PurchaseTicketRequest

Campos mínimos:

```
userEmail  
eventCode  
type
```

El precio no debería confiarse directamente desde el cliente. La estrategia de precio debe definirse dentro del sistema.

23. TicketResponse

Debe contener:

```
id  
ticketCode  
type  
price  
status  
purchaseDate  
userEmail  
eventCode  
eventName
```

No debe exponer directamente:

```
User  
Event
```

24. Flujo de compra

```
PurchaseTicketRequest
  ↓
Buscar User
  ↓
Validar User
  ↓
Buscar Event
  ↓
Validar Event
  ↓
Validar edad
  ↓
Validar capacidad
  ↓
Calcular precio
  ↓
Crear Ticket
  ↓
Guardar
  ↓
Actualizar SOLD_OUT si aplica
  ↓
Mapear Response
```

25. Reglas de compra

BR-TICKET-001 — Usuario existente

Si no existe:

```
ResourceNotFoundException
```

BR-TICKET-002 — Usuario activo

Un usuario inactivo no puede comprar.

BR-TICKET-003 — Evento existente

Si el evento no existe:

```
ResourceNotFoundException
```

BR-TICKET-004 — Evento publicado

Solo se pueden comprar tickets cuando:

```
event.status == PUBLISHED
```

No se permiten compras para:

```
DRAFT  
SOLD_OUT  
CANCELLED  
FINISHED
```

BR-TICKET-005 — Fecha futura

No se puede comprar para un evento que ya ocurrió.

BR-TICKET-006 — Edad mínima

Cuando:

```
event.minimumAge > 0
```

el sistema debe calcular la edad usando:

```
UserProfile.birthDate
```

La edad debe evaluarse en la fecha del evento.

Si no cumple:

```
BusinessRuleException
```

26. Capacidad

El venue posee:

```
capacity
```

La regla básica es:

```
paidTickets < venue.capacity
```

BR-TICKET-007

Si:

```
paidTickets >= venue.capacity
```

no se permite otra compra.

BR-TICKET-008

Cuando una compra completa la capacidad:

```
paidTickets == venue.capacity
```

el evento debe cambiar a:

```
SOLD_OUT
```

Esto debe ocurrir en la misma transacción.

27. Estado inicial del ticket

Para esta versión simplificada, una compra válida genera:

```
PAID
```

Una versión futura podrá incorporar:

```
RESERVED
```

y pasarela de pagos.

28. Precio

Debe utilizarse:

```
BigDecimal
```

BR-TICKET-009

Nunca se permite:

```
price < 0
```

Una implementación académica puede definir reglas como:

```
GENERAL    → precio base  
STUDENT    → descuento  
VIP        → multiplicador  
BACKSTAGE  → multiplicador superior
```

La estrategia escogida debe estar encapsulada y probada.

29. Cancelación

`cancel(ticketCode)` debe permitir:

```
PAID → CANCELLED
```

BR-TICKET-010

Solo puede cancelarse un ticket **PAID**.

BR-TICKET-011

No puede cancelarse:

```
USED  
CANCELLED
```

BR-TICKET-012

No puede cancelarse después de la fecha del evento.

30. Uso del ticket

`markAsUsed(ticketCode)` debe permitir:

PAID → USED

BR-TICKET-013

Solo puede marcarse como usado un ticket **PAID**.

BR-TICKET-014

Un ticket **CANCELLED** nunca puede utilizarse.

31. Ciclos de estado

Evento

DRAFT
↓
PUBLISHED
↓
SOLD_OUT
↓
FINISHED

Transiciones adicionales:

DRAFT → CANCELLED
PUBLISHED → CANCELLED

Ticket

PAID
├→ USED
└→ CANCELLED

32. Repositories requeridos

Ejemplos de operaciones necesarias:

VenueRepository

```
Optional<Venue> findByCode(String code);

List<Venue> findByActiveTrueOrderByNameAsc();
```

EventRepository

```
Optional<Event> findByEventCode(String eventCode);

boolean existsByEventCode(String eventCode);

List<Event> findByStatusOrderByEventDateAsc(
    EventStatus status
);
```

ArtistRepository

```
Optional<Artist> findByStageNameIgnoreCase(
    String stageName
);

List<Artist> findByActiveTrueOrderByStageNameAsc();
```

UserRepository

```
Optional<User> findByEmailIgnoreCase(String email);

Optional<User> findByUsername(String username);

boolean existsByEmailIgnoreCase(String email);

boolean existsByUsername(String username);
```

TicketRepository

```
Optional<Ticket> findByTicketCode(String ticketCode);

List<Ticket>
findByUserEmailIgnoreCaseOrderByPurchaseDateDesc(
```

```
        String email
    );

    long countByEventEventCodeAndStatus(
        String eventCode,
        TicketStatus status
    );
```

Las firmas pueden variar si mantienen el comportamiento solicitado.

33. MapStruct

Debe utilizarse MapStruct para:

```
Venue → VenueResponse
Event → EventResponse / EventSummaryResponse
Artist → ArtistResponse
User → UserResponse
Ticket → TicketResponse
```

Ejemplo:

```
@Mapper(componentModel = "spring")
public interface EventMapper {

    @Mapping(
        target = "venueCode",
        source = "venue.code"
    )
    @Mapping(
        target = "venueName",
        source = "venue.name"
    )
    EventResponse toResponse(Event event);
}
```

34. TicketMapper

Debe mapear:

```
ticket.user.email
    ↓
```

```
userEmail
```

```
ticket.event.eventCode  
    ↓  
eventCode
```

```
ticket.event.name  
    ↓  
eventName
```

35. Excepciones

Crear al menos:

```
ResourceNotFoundException  
BusinessRuleException  
DuplicateResourceException
```

ResourceNotFoundException

Cuando el recurso no existe.

Ejemplo:

```
Event not found: CMF-2026
```

DuplicateResourceException

Cuando existe conflicto de unicidad.

Ejemplo:

```
Username already exists.
```

BusinessRuleException

Cuando el recurso existe pero la operación no es válida.

Ejemplo:

```
User does not meet minimum age.
```

36. Optional

Preferir:

```
repository
    .findByEventCode(code)
    .orElseThrow(...)
```

No utilizar directamente:

```
optional.get()
```

sin comprobar su contenido.

37. Programación funcional

Para transformar listas:

```
return events
    .stream()
    .map(eventMapper::toSummary)
    .toList();
```

Debe utilizarse cuando mejore la claridad del código.

38. Transacciones

Lecturas como:

```
findByCode
findPublishedEvents
findByUserEmail
findPaidTicketsByEvent
```

pueden utilizar:

```
@Transactional(readOnly = true)
```

Escrituras como:

```
create  
publish  
addArtist  
register  
purchase  
cancel  
markAsUsed
```

deben utilizar:

```
@Transactional
```

39. Atomicidad de compra

La compra debe ser una operación atómica:

```
validar usuario  
+  
validar evento  
+  
validar edad  
+  
validar capacidad  
+  
calcular precio  
+  
crear ticket  
+  
actualizar SOLD_OUT
```

Si algún paso falla:

```
ROLLBACK
```

40. Requisitos funcionales

ID	Requisito
FR-SVC-001	Consultar venue por código
FR-SVC-002	Consultar venues activos
FR-SVC-003	Crear evento
FR-SVC-004	Consultar evento por código
FR-SVC-005	Consultar eventos publicados
FR-SVC-006	Publicar evento
FR-SVC-007	Asociar artista a evento
FR-SVC-008	Consultar eventos por artista
FR-SVC-009	Consultar artista
FR-SVC-010	Registrar usuario con perfil
FR-SVC-011	Consultar usuario por email
FR-SVC-012	Consultar usuario por username
FR-SVC-013	Comprar ticket
FR-SVC-014	Consultar ticket por código
FR-SVC-015	Consultar tickets de usuario
FR-SVC-016	Consultar tickets pagados de evento
FR-SVC-017	Cancelar ticket
FR-SVC-018	Marcar ticket como usado

41. Requisitos no funcionales

NFR-001 — Testabilidad

Los servicios deben poder probarse sin:

```
PostgreSQL
Spring ApplicationContext
Testcontainers
```

NFR-002 — Constructor injection

Toda dependencia debe inyectarse mediante constructor.

NFR-003 — DTOs inmutables

Los DTOs deben implementarse preferiblemente mediante **record**.

NFR-004 — Mapeo

Entity → DTO debe realizarse mediante MapStruct.

NFR-005 — Errores significativos

Los mensajes deben identificar el recurso o regla.

NFR-006 — Separación de responsabilidades

No debe existir:

- SQL dentro de Service;
- reglas de negocio dentro de Repository;
- entidades expuestas como contrato de Service.

42. Estrategia de pruebas

Las pruebas de Service serán:

UNIT TESTS

Arquitectura:

```
JUnit
  ↓
Service real
  ├── Repository Mock
  └── Mapper Mock
```

Utilizar:

```
JUnit 5
Mockito
AssertJ
```

Base:

```
@ExtendWith(MockitoExtension.class)
class EventServiceImplTest {
}
```

No utilizar `@SpringBootTest` para estos unit tests.

43. Tests mínimos de EventService

TEST-EVENT-001

Evento existente → retorna DTO.

TEST-EVENT-002

Evento inexistente → `ResourceNotFoundException`.

TEST-EVENT-003

Crear evento válido → `save()` ejecutado.

TEST-EVENT-004

Venue inexistente → error y `save()` nunca ejecutado.

TEST-EVENT-005

Venue inactivo → `BusinessRuleException`.

TEST-EVENT-006

Fecha pasada → `BusinessRuleException`.

TEST-EVENT-007

Publicar `DRAFT` válido → `PUBLISHED`.

TEST-EVENT-008

Publicar `CANCELLED` → `BusinessRuleException` y no persistir.

44. Tests mínimos de UserService

TEST-USER-001

Registrar usuario válido.

TEST-USER-002

Username duplicado → `DuplicateResourceException`.

TEST-USER-003

Email duplicado → `DuplicateResourceException`.

TEST-USER-004

Birth date futura → `BusinessRuleException`.

45. Tests mínimos de TicketService

TEST-TICKET-001

Compra válida → ticket `PAID`.

TEST-TICKET-002

Usuario inexistente → `ResourceNotFoundException`.

TEST-TICKET-003

Usuario inactivo → `BusinessRuleException`.

TEST-TICKET-004

Evento `DRAFT` → `BusinessRuleException`.

TEST-TICKET-005

Evento `CANCELLED` → `BusinessRuleException`.

TEST-TICKET-006

Usuario menor de edad → `BusinessRuleException`.

TEST-TICKET-007

Evento sin capacidad → `BusinessRuleException`.

TEST-TICKET-008

Último ticket disponible → guardar ticket y cambiar evento a `SOLD_OUT`.

TEST-TICKET-009

Cancelar ticket `PAID` → `CANCELLED`.

TEST-TICKET-010

Cancelar ticket **USED** → **BusinessRuleException**.

TEST-TICKET-011

Marcar **PAID** como usado → **USED**.

TEST-TICKET-012

Usar ticket **CANCELLED** → **BusinessRuleException**.

46. Mockito

Los tests deben demostrar uso de:

```
when(...)
verify(...)
verify(..., never())
any()
eq()
```

cuando corresponda.

Cada prueba debe seguir:

```
ARRANGE
ACT
ASSERT
```

47. Escenario principal de aceptación

Venue

```
Code: VEN-SMR-01
Name: Marina Convention Center
City: Santa Marta
Capacity: 3
Active: true
```

Event

Code: CMF-2026
Name: Caribbean Music Fest 2026
Status inicial: DRAFT
Category: MUSIC
Minimum Age: 18

Artists

Solar Beat
Neon Waves
Caribbean Sound

Users

Andrea

Email: andrea@email.com
Edad al momento del evento: 25
Active: true

Carlos

Email: carlos@email.com
Edad: 21
Active: true

Laura

Email: laura@email.com
Edad: 17
Active: true

Miguel

Email: miguel@email.com
Edad: 30
Active: false

48. Criterios de aceptación

AC-001

CMF-2026 puede crearse y debe iniciar DRAFT.

AC-002

Puede publicarse cuando las reglas se cumplen.

AC-003

Se pueden asociar los tres artistas.

AC-004

Andrea puede comprar.

AC-005

Carlos puede comprar.

AC-006

Laura no puede comprar por edad mínima.

AC-007

Miguel no puede comprar porque está inactivo.

AC-008

Un tercer usuario adulto puede adquirir el último ticket y el evento pasa a SOLD_OUT.

AC-009

Una cuarta compra debe producir BusinessException.

AC-010

Un ticket PAID puede pasar a USED.

AC-011

Un ticket USED no puede cancelarse.

49. Matriz de trazabilidad

Requisito	Servicio	Reglas	Tests
FR-SVC-003	EventService	BR-EVENT-001..006	TEST-EVENT-003..006
FR-SVC-006	EventService	BR-EVENT-007..009	TEST-EVENT-007..008
FR-SVC-007	EventService	BR-EVENT-010..011	Unit test requerido
FR-SVC-010	UserService	BR-USER-001..005	TEST-USER-001..004
FR-SVC-013	TicketService	BR-TICKET-001..009	TEST-TICKET-001..008
FR-SVC-017	TicketService	BR-TICKET-010..012	TEST-TICKET-009..010
FR-SVC-018	TicketService	BR-TICKET-013..014	TEST-TICKET-011..012

50. Definition of Done

- [] Existen interfaces Service
- [] Existen implementaciones @Service
- [] Se utiliza constructor injection
- [] Los DTOs utilizan record
- [] No se exponen entidades JPA
- [] MapStruct transforma Entity → DTO
- [] Existen excepciones personalizadas
- [] Optional se maneja de forma segura
- [] Las reglas están en Service
- [] Las escrituras son transaccionales
- [] Las lecturas utilizan readOnly cuando corresponde
- [] VenueService implementado
- [] ArtistService implementado
- [] EventService implementado
- [] UserService implementado
- [] TicketService implementado
- [] Compra valida usuario

- [] Compra valida evento
- [] Compra valida edad
- [] Compra valida capacidad
- [] SOLD_OUT se actualiza correctamente
- [] Unit tests usan Mockito
- [] Unit tests usan AssertJ
- [] No se levanta PostgreSQL en unit tests
- [] No se utiliza @SpringBootTest para tests unitarios de Service
- [] Se utiliza verify(..., never()) en caminos inválidos
- [] mvn clean test finaliza correctamente

51. Preguntas de sustentación

1. ¿Cuál es la responsabilidad de Service?
2. ¿Qué diferencia existe entre Service y Repository?
3. ¿Por qué Service no debería retornar entidades JPA?
4. ¿Qué ventaja tiene `record` para DTOs?
5. ¿Qué hace MapStruct?
6. ¿Por qué preferir constructor injection?
7. ¿Qué diferencia existe entre `ResourceNotFoundException` y `BusinessRuleException`?
8. ¿Por qué evitar `Optional.get()`?
9. ¿Cuándo utilizar `@Transactional(readOnly = true)`?
10. ¿Cuándo utilizar `@Transactional`?
11. ¿Por qué una compra debe ser atómica?
12. ¿Por qué la capacidad pertenece a una regla de Service?
13. ¿Por qué el cálculo de edad pertenece al negocio?
14. ¿Por qué utilizar `BigDecimal` para precios?
15. ¿Qué diferencia existe entre unit test e integration test?
16. ¿Por qué Mockito permite probar Service sin PostgreSQL?
17. ¿Qué demuestra `verify(repository, never()).save(...)`?
18. ¿Por qué `SOLD_OUT` debe actualizarse dentro de la misma transacción?
19. ¿Por qué Ticket es una entidad y no solo una relación N:M?
20. ¿Qué reglas moverías a componentes especializados si PulsePass creciera?

52. Evolución futura

Una siguiente versión podrá incorporar:

```
PricingService
CapacityService
RecommendationService
PaymentService
NotificationService
```

También:

- promociones;
- descuentos;
- reservas temporales;
- reembolsos;
- precios múltiples;
- asientos numerados;
- optimistic locking;
- eventos de dominio;
- Kafka;
- pagos externos.

53. Consideración sobre concurrencia

La estrategia:

```
contar tickets
  ↓
validar capacity
  ↓
save ticket
```

es suficiente para el ejercicio académico.

En producción, dos compras concurrentes podrían observar el mismo cupo disponible.

Una versión posterior deberá analizar:

```
Optimistic Locking
Pessimistic Locking
Atomic updates
Database constraints
Distributed concurrency
```

Este problema queda fuera del alcance de esta versión.

54. Competencia final esperada

Ante:

Andrea quiere comprar una entrada VIP para CMF-2026.

el flujo mental esperado es:

```
PurchaseTicketRequest
  ↓
TicketService
  ↓
Buscar Andrea
  ↓
¿Existe?
  ↓
¿Está activa?
  ↓
Buscar CMF-2026
  ↓
¿Existe?
  ↓
¿PUBLISHED?
  ↓
¿Fecha válida?
  ↓
¿Cumple edad?
  ↓
¿Hay capacidad?
  ↓
Calcular precio
  ↓
Crear Ticket
  ↓
Repository.save()
  ↓
¿Se agotó capacidad?
  ↓
Actualizar SOLD_OUT
  ↓
TicketMapper
  ↓
TicketResponse
```

La capa Service transforma operaciones técnicas de persistencia en **comportamiento coherente de negocio**.